



INDIAN INSTITUTE OF TECHNOLOGY DELHI

COL 729 Lab 2: IR and Compiler Passes

Submitted by

Aditya Anand and Varun Subramaniam
2023CS50284 and 2023CS50497

Instructor

Prof. Sorav Bansal

9 September 2026

1 Opcodes

All except `select` are taken from `opcodes.00.ll`. `select` is from `opcodes.canonical.ll`.

alloca

```
13: %1 = alloca i32, align 4
```

This line creates a space for a 32-bit integer on the stack (for `compiletimeindex`) and `%1` is its address. The result type is `ptr`.

load

```
16: %3 = load volatile i32, ptr @runtimeindex, align 4
```

This line loads a 32-bit integer *from* `@runtimeindex` and writes that value into the SSA name `%3`. It does not store into `runtimeindex`. The result type is `i32`.

store

```
15: store i32 0, ptr %1, align 4
```

This stores the value 0 into the memory that `%1` points to (`compiletimeindex`). `%1` is only the address and it does not change. `store` has no result.

getelementptr

```
18: %5 = getelementptr inbounds [2 x %struct.myStruct], ptr %2, i64 0, i64 %4
```

`%2` gives the base address of the `myStruct` array. The first `i64 0` moves the pointer by 0 distance forward and changes the pointing type from a hypothetical array of `[2 x %struct.myStruct]` and points to a `[2 x %struct.myStruct]` without moving. The second `i64 %4` does pointer arithmetic with the offset stored in `%4` and adds that as offset, so `%5` is the required address. The result type is `ptr`. This is done in the line `arr[runtimeindex].scalar = 1`. The `getelementptr` arithmetic is purely done on the addresses. There is no memory access involved. The memory access is done by a separate load/store. Hence, `getelementptr` does not read the memory.

icmp

```
46: %5 = icmp slt i32 %4, 3
```

This is for integer comparison. It sets the `%5` value to true if the `%4` value is less than 3. This is used to check the breaking condition of the loop in `f2`. The result type is `i1`.

fcmp

```
67: %18 = fcmp oeq double %17, 2.000000e+00
```

`oeq` means that none of the numbers being compared is a NaN and that both the values being compared are equal. This line checks if `%17` is equal to 2.0f to check for the break condition in the loop in `f2`. The result type is `i1`.

add

```
51: %8 = add nsw i32 %7, 1
```

nsw means no signed wrap, meaning that no overflow will take place. In this line 1 is added to %7 to get %8.

This is the `loopcarried += 1` step inside the loop in `f2`. %7 is the value loaded from `loopcarried` on line 50. The result type is `i32`.

fdiv

```
55: %11 = fdiv float %10, 2.000000e+00
```

The float division value of %10 divided by 2.0f is stored in %11 as done in `f2`. The result type is `float`.

br

```
42: br label %3
```

This jumps the eip of the program to the basic block started by label 3. `br` has no result.

switch

```
96: switch i32 %7, label %29 [
```

It uses the value of %7 as the switch condition in `f3`. If it does not match with anything then the eip changes to %29. `switch` has no result.

phi

```
120: %16 = phi i32 [ %13, %12 ], [ 7, %14 ]
```

It changes this line `int t = (runtimeindex > 1) ? loopcarried : 7;`. Clang emits it as 4 basic blocks %9, %12, %14, %15; the phi does not create them. The phi sits at the join block %15 and picks %13 if control came from %12 and 7 if it came from %14, so that SSA is not broken. The result type is `i32`.

call

```
104: call void @f1()
```

Calls the function `f1`. It is the first case of the switch. `f1` returns void so there is no result.

ret

```
34: ret void
```

It returns void at the end of `f1`. `ret` has no result.

sext

```
17: %4 = sext i32 %3 to i64
```

It is a **sext** (signed extension) of the value of %3 from 32 to 64 bits because pointer arithmetic is done in 64 bits. The result type is **i64**.

sitofp

```
54: %10 = sitofp i32 %9 to float
```

It is used in this line `float myf = loopcarried/2.0f;`. %9 is the `loopcarried` value, which is converted into a float register %10 before doing arithmetic. The result type is **float**.

select

```
71: %4 = select i1 %2, i32 %3, i32 7
```

It changes this line `int t = (runtimeindex > 1) ? loopcarried : 7;`. Instead of using 2 basic blocks to first calculate the values and using phi, it is storing the result of the comparison in %2 and using its result directly to take from one of the branches. The result type is **i32**.

2 Control Flow Graph

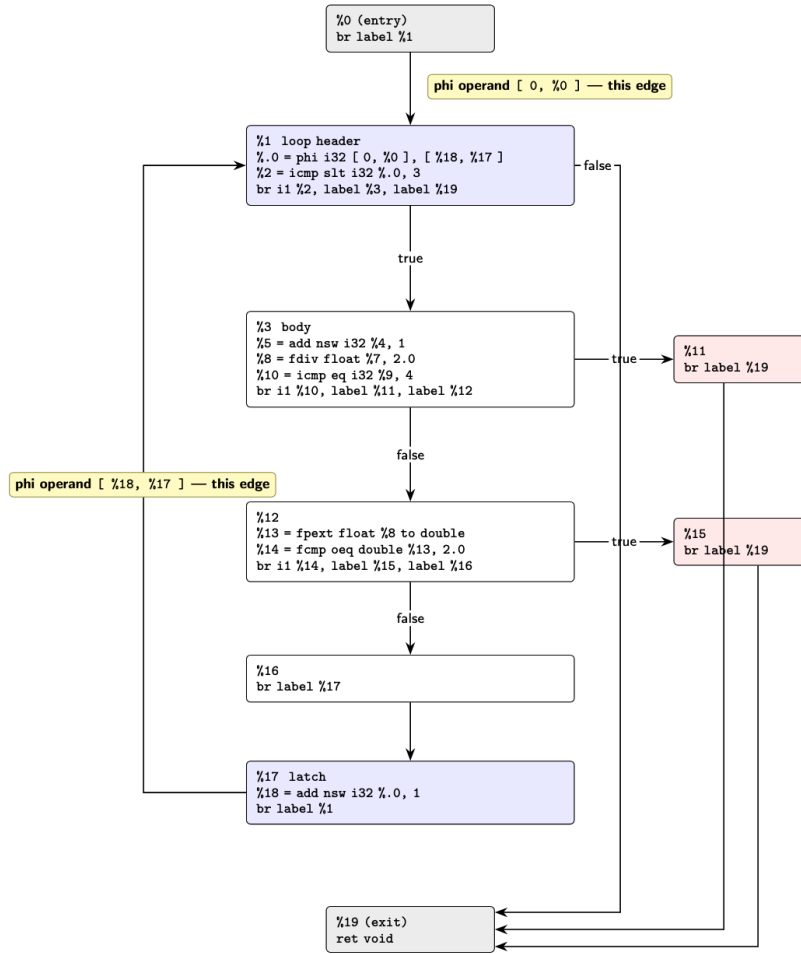


Figure 1: Control flow graph.

3 opcodes.mem2reg.ll

Explaining struct access

```

%5 = getelementptr @inbounds, [2 x %struct.myStruct], ptr %2, i64 0, i64 %4
%6 = getelementptr @inbounds, nuw %struct.myStruct, ptr %5, i32 0, i32 0

```

The layout of `myStruct` on this target is: `scalar` at offset 0, `array[2][2]` at bytes 4–19, 4 bytes of padding so the pointer is aligned, `ptr` at offset 24, total size 32, alignment 8.

First GEP: `%2` is the base of the `arr` allocation. `i64 0` treats that pointer as pointing at one `[2 x %struct.myStruct]` and does not move it. `i64 %4` is the run-time index `runtimeindex`. That steps by the `struct` size, so the address is `base + 32 * runtimeindex`. Result: pointer to `arr[runtimeindex]`.

Second GEP: the first `i32 0` is “this struct, no extra stride”. The second `i32 0` is field 0, which is `.scalar`. That field is at offset 0, not $32 \times$ anything. 32 bytes is only the size of one `myStruct`, used by the array-element index above.

The getelementptr arithmetic is purely done on the addresses. There is no memory access involved. The memory access is done by a separate load/store. Hence, getelementptr does not read the memory.

4 Part B

function	opt	blocks	br	ret	switch	add	alloca	call	fcmp	fdiv	fmul	fpext	getelementptr	icmp	load	phi	select	sext	sitofp	store	trunc
f1	-O0	1	0	1	0	0	2	0	0	0	0	0	9	0	3	0	0	3	0	4	0
f1	-O1	1	0	1	0	0	1	2	0	0	0	0	3	0	3	0	0	3	0	3	0
f1	-O2	1	0	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
f1	-O3	1	0	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
f2	-O0	9	8	1	0	2	2	0	1	1	0	1	0	2	6	0	0	0	1	4	0
f2	-O1	5	4	1	0	3	0	0	1	0	1	0	0	2	1	3	0	0	1	1	0
f2	-O2	6	5	1	0	3	0	0	2	0	2	0	0	2	1	1	0	0	2	1	0
f2	-O3	6	5	1	0	3	0	0	2	0	2	0	0	2	1	1	0	0	2	1	0
f3	-O0	9	7	1	1	1	6	2	0	0	0	0	0	1	9	1	0	1	1	8	1
f3	-O1	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
f3	-O2	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
f3	-O3	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
main	-O0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O2	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O3	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 2: Opcode counts per function and optimisation level.

1)

Before:

```
%1 = alloca i32, align 4
store i32 0, ptr %1, align 4
br label %3

3:
%4 = load i32, ptr %1, align 4
%5 = icmp slt i32 %4, 3
br i1 %5, label %6, label %24

21:
%22 = load i32, ptr %1, align 4
%23 = add nsw i32 %22, 1
store i32 %23, ptr %1, align 4
br label %3, !llvm.loop !5
```

After opt -S -passes=mem2reg opcodes.00.11:

```
br label %1

1:
%.0 = phi i32 [ 0, %0 ], [ %18, %17 ]
%2 = icmp slt i32 %.0, 3
...

17:
%18 = add nsw i32 %.0, 1
br label %1, !llvm.loop !5
```

Explanation: Change made by mem2reg. In f2, the loop counter becomes a phi node. Earlier the loop counter was in the stack slot. Now it is promoted to a register.

2)

f3 switches on int a = 1. At -00 the switch and all four cases are still there (9 blocks, one switch). Cases 0, 2 and 3 are dead.

Before (opcodes.00.11):

```
store i32 1, ptr %1, align 4
%6 = load i32, ptr %1, align 4
switch i32 %6, label %31 [
    i32 0, label %7
    i32 1, label %8
    i32 2, label %19
    i32 3, label %25
]
```

After opt -S -passes='function(mem2reg,sccp,simplifcfg)':

```
%1 = load volatile i32, ptr @runtimeindex, align 4
%2 = icmp sgt i32 %1, 1
%3 = load i32, ptr @loopcarried, align 4
%4 = select i1 %2, i32 %3, i32 7
%5 = load i32, ptr @loopcarried, align 4
%6 = add nsw i32 %5, %4
store i32 %6, ptr @loopcarried, align 4
call void @f1()
ret void
```

sccp sees that the switch condition is the constant 1. simplifcfg deletes the unused arms and the switch. What is left is only case 1: the ternary becomes a select, then call @f1. That is why the Part B table goes from 9 blocks to 1 and from one switch to zero at -01. This is not mem2reg; mem2reg alone still leaves the four cases.

3)

Before:

```
3:                                     ; preds = %1
%9 = load i32, ptr @loopcarried, align 4
%10 = icmp eq i32 %9, 4
br i1 %10, label %11, label %12

11:                                   ; preds = %3
br label %19

12:                                   ; preds = %3
%13 = fpext float %8 to double
%14 = fcmp oeq double %13, 2.000000e+00
br i1 %14, label %15, label %16

15:                                   ; preds = %12
br label %19

16:                                   ; preds = %12
br label %17
```

After:

```

3:                                     ; preds = %1
%9 = load i32, ptr @loopcarried, align 4
%10 = icmp eq i32 %9, 4
%11 = fpext float %8 to double
%12 = fcmp oeq double %11, 2.000000e+00
%or.cond = select i1 %10, i1 true, i1 %12
br i1 %or.cond, label %15, label %13

```

The before code is after `opt -S -passes=mem2reg opcodes.00.ll`. The after code is after `opt -S -passes='mem2reg,simplifcfg' opcodes.00.ll`.

It collapses the 2 statements in the code:

```

if (loopcarried == 4) break;
else if (myf == 2.0) break;

```

into a common `or.cond` check, simplifying the branch specification.

5 Part C

These numbers are from Homebrew LLVM 18.1.8 on arm64 macOS (`opt -version`), the same install used for clang.

The pipeline was dumped with `opt -passes="default<0n>" -print-pipeline-passes`. Names followed by (...) are adaptors (function, cgsc, loop, loop-mssa, devirt, coro-cond) and were dropped. `require<>`, `invalidate<>`, `recompute-globalsaa`, `verify`, `annotation-remarks`, `transform-warning` and `cg-profile` were dropped because they are analyses or instrumentation. `AArch64LoopIdiomTransformPass` showed up in the dump but it is a C++ class name and cannot be written in `-passes='...'`, so it was skipped. Repeats were kept, in order.

Counts

level	invocations	distinct
-00	5	5
-01	81	55
-02	97	66
-03	100	69

Ordered lists

Same order as the dump, one column per level. Type is small so each name stays on one line. What each name does is in the table below; the numbers there are how many times it ran.

-00	-01	-02	-03
1. always-inline	1. annotation2metadata	1. annotation2metadata	1. annotation2metadata
2. coro-early	2. forceattrs	2. forceattrs	2. forceattrs
3. coro-split	3. inferattrs	3. inferattrs	3. inferattrs
4. coro-cleanup	4. coro-early	4. coro-early	4. coro-early
5. globaldce	5. lower-expect	5. lower-expect	5. lower-expect
	6. simplifcfg	6. simplifcfg	6. simplifcfg
	7. sroa	7. sroa	7. sroa
	8. early-cse	8. early-cse	8. early-cse
	9. openmp-opt	9. openmp-opt	9. callsite-splitting
	10. ipscpp	10. ipscpp	10. openmp-opt
	11. called-value-propagation	11. called-value-propagation	11. ipscpp
	12. globalopt	12. globalopt	12. called-value-propagation
	13. mem2reg	13. mem2reg	13. globalopt
	14. instcombine	14. instcombine	14. mem2reg
	15. simplifcfg	15. simplifcfg	15. instcombine
	16. always-inline	16. always-inline	16. simplifcfg
	17. inline	17. inline	17. always-inline

-00	-01	-02	-03
	18. function-attrs	18. function-attrs	18. inline
	19. sroa	19. openmp-opt-cgsc	19. function-attrs
	20. early-cse	20. sroa	20. argpromotion
	21. simplifycfg	21. early-cse	21. openmp-opt-cgsc
	22. instcombine	22. speculative-execution	22. sroa
	23. libcalls-shrinkwrap	23. jump-threading	23. early-cse
	24. simplifycfg	24. correlated-propagation	24. speculative-execution
	25. reassociate	25. simplifycfg	25. jump-threading
	26. loop-instsimplify	26. instcombine	26. correlated-propagation
	27. loop-simplifycfg	27. aggressive-instcombine	27. simplifycfg
	28. licm	28. libcalls-shrinkwrap	28. instcombine
	29. loop-rotate	29. tailcallelim	29. aggressive-instcombine
	30. licm	30. simplifycfg	30. libcalls-shrinkwrap
	31. simple-loop-unswitch	31. reassociate	31. tailcallelim
	32. simplifycfg	32. constraint-elimination	32. simplifycfg
	33. instcombine	33. loop-instsimplify	33. reassociate
	34. loop-idiom	34. loop-simplifycfg	34. constraint-elimination
	35. indvars	35. licm	35. loop-instsimplify
	36. loop-deletion	36. loop-rotate	36. loop-simplifycfg
	37. loop-unroll-full	37. licm	37. licm
	38. sroa	38. simple-loop-unswitch	38. loop-rotate
	39. memcpypopt	39. simplifycfg	39. licm
	40. sccp	40. instcombine	40. simple-loop-unswitch
	41. bdce	41. loop-idiom	41. simplifycfg
	42. instcombine	42. indvars	42. instcombine
	43. coro-elide	43. loop-deletion	43. loop-idiom
	44. adce	44. loop-unroll-full	44. indvars
	45. simplifycfg	45. sroa	45. loop-deletion
	46. instcombine	46. vector-combine	46. loop-unroll-full
	47. function-attrs	47. mldst-motion	47. sroa
	48. coro-split	48. gvn	48. vector-combine
	49. deadargelim	49. sccp	49. mldst-motion
	50. coro-cleanup	50. bdce	50. gvn
	51. globalopt	51. instcombine	51. sccp
	52. globaldce	52. jump-threading	52. bdce
	53. elim-avail-extern	53. correlated-propagation	53. instcombine
	54. rpo-function-attrs	54. adce	54. jump-threading
	55. float2int	55. memcpypopt	55. correlated-propagation
	56. lower-constant-intrinsics	56. dse	56. adce
	57. loop-rotate	57. move-auto-init	57. memcpypopt
	58. loop-deletion	58. licm	58. dse
	59. loop-distribute	59. coro-elide	59. move-auto-init
	60. inject-tli-mappings	60. simplifycfg	60. licm
	61. loop-vectorize	61. instcombine	61. coro-elide
	62. infer-alignment	62. function-attrs	62. simplifycfg
	63. loop-load-elim	63. coro-split	63. instcombine
	64. instcombine	64. deadargelim	64. function-attrs
	65. simplifycfg	65. coro-cleanup	65. coro-split
	66. vector-combine	66. globalopt	66. deadargelim
	67. instcombine	67. globaldce	67. coro-cleanup
	68. loop-unroll	68. elim-avail-extern	68. globalopt
	69. sroa	69. rpo-function-attrs	69. globaldce
	70. infer-alignment	70. float2int	70. elim-avail-extern
	71. instcombine	71. lower-constant-intrinsics	71. rpo-function-attrs
	72. licm	72. loop-rotate	72. float2int
	73. alignment-from-assumptions	73. loop-deletion	73. lower-constant-intrinsics
	74. loop-sink	74. loop-distribute	74. chr
	75. instsimplify	75. inject-tli-mappings	75. loop-rotate
	76. div-rem-pairs	76. loop-vectorize	76. loop-deletion
	77. tailcallelim	77. infer-alignment	77. loop-distribute
	78. simplifycfg	78. loop-load-elim	78. inject-tli-mappings
	79. globaldce	79. instcombine	79. loop-vectorize
	80. constmerge	80. simplifycfg	80. infer-alignment
	81. rel-lookup-table-converter	81. slp-vectorizer	81. loop-load-elim
		82. vector-combine	82. instcombine
		83. instcombine	83. simplifycfg
		84. loop-unroll	84. slp-vectorizer
		85. sroa	85. vector-combine
		86. infer-alignment	86. instcombine
		87. instcombine	87. loop-unroll
		88. licm	88. sroa
		89. alignment-from-assumptions	89. infer-alignment
		90. loop-sink	90. instcombine
		91. instsimplify	91. licm
		92. div-rem-pairs	92. alignment-from-assumptions
		93. tailcallelim	93. loop-sink
		94. simplifycfg	94. instsimplify
		95. globaldce	95. div-rem-pairs
		96. constmerge	96. tailcallelim
		97. rel-lookup-table-converter	97. simplifycfg
			98. globaldce
			99. constmerge
			100. rel-lookup-table-converter

Pass descriptions

pass	description	-00	-01	-02	-03
adce	deletes dead code even if it is only dead on some paths	0	1	1	1
aggressive-instcombine	extra instcombine work the cheap pass will not touch	0	0	1	1
alignment-from-assumptions	if an assume says the pointer is aligned, mark the load/store that way	0	1	1	1
always-inline	inlines <code>always_inline</code> functions; this is why it still runs at -00	1	1	1	1
annotation2metadata	turns the old annotation globals into metadata	0	1	1	1
argpromotion	if you only pass a pointer to read one field, just pass the field	0	0	0	1
bdce	DCE that also drops unused <i>bits</i> , not just unused values	0	1	1	1
called-value-propagation	if a call always hits the same function, put that name in	0	1	1	1
callsite-splitting	clones a call so each incoming edge can see a constant	0	0	0	1
chr	flattens a pile of hot ifs so the common path is shorter	0	0	0	1
constmerge	one copy of the same constant string / array	0	1	1	1
constraint-elimination	if an earlier icmp already proved this one, fold it	0	0	1	1
coro-cleanup	leftover tidy-up after coroutine lowering	1	1	1	1
coro-early	first coroutine rewrite	1	1	1	1
coro-elide	skip the coro heap alloc when the frame is small enough	0	1	1	1
coro-split	cuts the coroutine into resume / destroy / cleanup	1	1	1	1
correlated-propagation	you just branched on x, so later uses of x can be folded	0	0	2	2
deadargelim	drops arguments nobody reads	0	1	1	1
div-rem-pairs	keeps a matching div and rem together so both are not done from scratch	0	1	1	1
dse	deletes a store that is overwritten or never read	0	0	1	1
early-cse	CSE, but early, before the loop passes	0	2	2	2
elim-avail-extern	after inlining an <code>available_externally</code> body, delete what is left	0	1	1	1
float2int	rewrite a float loop that is really just integer work	0	1	1	1
forceattrs	attributes you asked for on the command line	0	1	1	1
function-attrs	figures out <code>readonly</code> , <code>nocapture</code> , that kind of thing	0	2	2	2
globaldce	deletes internal globals that nothing references	1	2	2	2
globalopt	folds / shrinks globals and global constructors	0	2	2	2
gvn	numbers values so redundant loads and exprs disappear	0	0	1	1

pass	description	-00	-01	-02	-03
indvars	puts the loop index into a form later passes like	0	1	1	1
infer-alignment	guesses a better alignment for loads and stores	0	2	2	2
inferattrs	known attrs on things like <code>memcpy</code> / <code>printf</code>	0	1	1	1
inject-tli-mappings	tells the vectoriser which libcalls have a vector version	0	1	1	1
inline	the normal inliner	0	1	1	1
instcombine	the usual peephole / algebraic cleanup; they rerun this a lot	0	8	8	8
instsimplify	like instcombine but it will not invent new instructions	0	1	1	1
ipsccp	constant prop across functions, including killing dead calls	0	1	1	1
jump-threading	if you already know which way a block branches, jump straight there	0	0	2	2
libcalls-shrinkwrap	cheap check in front of a libcall, e.g. <code>isnan</code> before <code>log</code>	0	1	1	1
licm	pull work that does not change out of the loop	0	3	4	4
loop-deletion	delete a loop that does not actually do anything	0	2	2	2
loop-distribute	split a loop so the vectoriser can take one half	0	1	1	1
loop-idiom	turn a copy/set loop into <code>memcpy/memset</code>	0	1	1	1
loop-instsimplify	instsimplify, but only inside the loop	0	1	1	1
loop-load-elim	store then load the same slot next iteration: just reuse the value	0	1	1	1
loop-rotate	while $\rightarrow$ do-while so LICM has a pre-header that always runs	0	2	2	2
loop-simplifycfg	<code>simplifycfg</code> on just the loop blocks	0	1	1	1
loop-sink	push a calc <i>into</i> the loop if that is actually cheaper	0	1	1	1
loop-unroll	unroll some of the trips (how many depends on O1/O2/O3)	0	1	1	1
loop-unroll-full	unroll the whole thing when the trip count is tiny	0	1	1	1
loop-vectorize	turn the loop into vector ops (at O1 this is forced-only)	0	1	1	1
lower-constant-intrinsics	expand <code>llvm.objectsize</code> and similar	0	1	1	1
lower-expect	<code>llvm.expect</code> becomes a branch-weight hint	0	1	1	1
mem2reg	allocas become SSA; this is the pass from part B	0	1	1	1
memcpyopt	glue a run of stores into a <code>memcpy/memset</code>	0	1	1	1
mldst-motion	same load/store on both sides of an if: hoist or sink it	0	0	1	1
move-auto-init	move the auto-init <code>memset</code> off the fast path	0	0	1	1
openmp-opt	OpenMP-specific opts; we do not have OpenMP in this file	0	1	1	1
openmp-opt-cgsc	same idea, at CGSCC scope	0	0	1	1

pass	description	-00	-01	-02	-03
reassociate	reshuffle + and * so CSE and LICM can see more	0	1	1	1
rel-lookup-table-converter	switch lookup tables stored as relative offsets	0	1	1	1
rpo-function-attrs	another attribute pass; walks functions bottom-up	0	1	1	1
sccp	if a value is constant on the paths that actually run, replace it	0	1	1	1
simple-loop-unswitch	clone the loop for each way an invariant if can go	0	1	1	1
simplifycfg	merge blocks and kill trivial branches; part B change 3	0	8	8	8
slp-vectorizer	vectorise straight-line code, not a loop	0	0	1	1
speculative-execution	do a cheap op before the branch instead of after	0	0	1	1
sroa	smash a struct/array alloca into separate scalars	0	4	4	4
tailcallelim	rewrite a tail call as a jump, or at least mark it tail	0	1	2	2
vector-combine	tidy vector ops after the vectorisers have run	0	1	2	2

1)

These five names are common to all four levels: `always-inline`, `coro-early`, `coro-split`, `coro-cleanup`, `globaldce`.

-00 is just `always-inline`, the coroutine lowering, and a global DCE. Nothing like `mem2reg` or `instcombine` runs there.

2)

First introduced at -O1. This is where the real pipeline starts: `mem2reg`, `sroa`, `instcombine`, `simplifycfg`, `inline`, `ipsccp`, the loop nest (`licm`, `loop-rotate`, `indvars`, `loop-unroll`, `loop-vectorize`), `sccp`, `adce`, and the rest of the O1 list above.

First introduced at -O2. `jump-threading`, `correlated-propagation`, `gvn`, `dse`, `slp-vectorizer`, `aggressive-instcombine`, `constraint-elimination`, `mldst-motion`, `speculative-execution`, `move-auto-init`, `openmp-opt-cgsc`.

First introduced at -O3. Only three new names: `callsite-splitting`, `argpromotion`, `chr`.

3)

`licm` goes from 3 times at -O1 to 4 times at -O2/-O3 because O2 puts an extra `licm` after `dse`.

`tailcallelim` is once at the end of O1 and twice at O2 (once in the CGSCC simplification and once at the end).

`vector-combine` is once at O1 and twice at O2 (after the mid-pipeline `sroa`, and again after `slp-vectorizer`).

`jump-threading` and `correlated-propagation` both run twice at O2/O3, once before the loop nest and once after it.

`memcpyopt` sits right after the first `sroa` at O1. At O2 it moves later, after `adce`, because `gvn` and jump threading run first.

Most other O1 passes keep the same count but get a later index at O2, because the extra CGSCC passes are inserted after the inliner.

`instcombine` and `simplifycfg` both run 8 times from O1 up. They keep getting re-run after other passes make new opportunities.

The dump also changes parameters even when the name is the same. `loop-unroll` is `loop-unroll<01>`, `<02>`, `<03>`, so it is more willing to unroll at higher levels. `loop-vectorize` at O1 is `vectorize-forced-only`; at O2/O3 it is not, so real vectorisation is an O2 thing. `simple-loop-unswitch` at O3 is `<nontrivial;trivial>` and at O1/O2 it is only trivial unswitch.

6 Part D

`f1-f4` are `noinline`. The length is a parameter. `restrict` is on every array argument. `main` allocates arrays of 4096 elements, calls all four kernels, and prints a checksum for $n \in \{7, 16, 1023, 1024, 2048\}$. 1024 and 2048 are at least 1024 elements; 7 and 1023 are not a multiple of the vector width.

The first compile is `-O3 -fno-vectorize -fno-slp-vectorize`. Those loops stay scalar: one `load` / `fmuladd` / `store` per float, and a scalar `phi` for the dot product.

The second compile is `-O3 -march=native` on arm64, so the vector registers are NEON. The remarks here are:

```
vector.c:7:  vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:17: vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:27: vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:37: vectorized loop (vectorization width: 4, interleaved count: 2)
```

Width 4 is NEON 128 bits / 32-bit float or i32. Interleave 4 means the body is unrolled four vector steps, so 16 elements per trip. The leftover tail stays scalar, which is how $n = 7$ and $n = 1023$ still work.

The vector types in the file are `<4 x float>`, `<4 x i32>`, `<4 x i1>` and `<8 x i32>`.

Tracing f1

```
c[i] = a[i] * myalpha + b[i].
```

`myalpha` is one scalar, so it is splatted into a `<4 x float>` before the loop:

```
%12 = insertelement <4 x float> poison, float %4, i64 0
%13 = shufflevector <4 x float> %12, <4 x float> poison,
      <4 x i32> zeroinitializer
```

`insertelement` writes `myalpha` into lane 0 of a poison vector. The shuffle mask is all zeros, so every lane becomes that same value. That is the broadcast.

Then each vector step is `load` / `fma` / `store` (four of these, because of the interleave):

```
%20 = load <4 x float>, ptr %16, align 4
%28 = load <4 x float>, ptr %24, align 4
%32 = call <4 x float> @llvm.fmuladd.v4f32(<4 x float> %20,
      <4 x float> %13,
```

```

                                <4 x float> %28)
store <4 x float> %32, ptr %36, align 4

```

%20 is four `a[i]`, %28 is four `b[i]`, %13 is the broadcast `myalpha`. The result is four `c[i]` values stored together. No memory is read by the shuffle; the loads and the store are the memory ops.

Dot product and `llvm.vector.reduce`

The integer reduction is `loopcarried += a[i] * b[i]`. Four `<4 x i32>` partial sums run around the loop (the `interleave`). After the loop they are added together and collapsed with

```
%46 = call i32 @llvm.vector.reduce.add.v4i32(<4 x i32> %45)
```

That intrinsic is the horizontal add. There is no `extractelement` tree. The leftover iterations use a scalar `phi`, same as the `-fno-vectorize` file.

Blend: the loop that did not vectorise

The first version was

```
c[i] = mypred[i] ? a[i] : b[i];
```

The remark was “the cost-model indicates that vectorization is not beneficial”, and only interleaving happened. Clang selected the `a` or `b` *pointer* and then did a scalar load, so there was no `<4 x i32>` select.

The loop was changed to load both values first:

```
int left = a[i];
int right = b[i];
c[i] = mypred[i] ? left : right;
```

Now it vectorises. One step is

```
%18 = load <4 x i32>, ptr %14
%26 = load <4 x i32>, ptr %22
%34 = load <4 x i32>, ptr %30
%38 = icmp eq <4 x i32> %34, zeroinitializer
%42 = select <4 x i1> %38, <4 x i32> %26, <4 x i32> %18
store <4 x i32> %42, ptr %46
```

%34 is four predicates. `icmp eq` against zero gives a `<4 x i1>`. `select` picks `b` when the predicate is 0 and `a` otherwise, which is the C ternary. Then four results are stored.

Vector-specific opcodes

`insertelement`. Only the `myalpha` splat in `f1`, above.

`shufflevector`. Two uses. The all-zero mask is the broadcast. The other one is `f4 (c[2*i]=a[i]; c[2*i+1]=b[i];)`:

```
%29 = shufflevector <4 x i32> %17, <4 x i32> %23,
      <8 x i32> <i32 0, i32 4, i32 1, i32 5, i32 2, i32 6, i32 3, i32 7>
store <8 x i32> %29, ptr %27
```

Lanes 0..3 of the first vector are **a**, lanes 0..3 of the second are **b**. The mask 0,4,1,5,... is **a0,b0,a1,b1,...**. The result type is **<8 x i32>** so eight ints are stored in one go.

extractelement. Not emitted. The reduction uses **llvm.vector.reduce.add.v4i32** instead of pulling lanes out one by one. The interleave loop was added so that a real shuffle would show up; it did not produce **extractelement** either.

select. The vector **select <4 x i1>** in **f3**, above.

Correctness

Both binaries were built from this **vector.c**:

```
clang -O3 -fno-vectorize -fno-slp-vectorize vector.c -o vector.scalar
clang -O3 -march=native vector.c -o vector.vector
./vector.scalar
./vector.vector
```

They printed the same **n** and checksum for the five sizes in **main**. The checksum casts each float to **int** before hashing, so a last-bit FMA difference between the scalar and vector builds would not show up. That is intentional: contraction can change the last bits without meaning the kernels disagree.

The printed values were:

```
7 2301122756
16 3179001926
1023 2561569046
1024 4266366105
2048 2497880110
```

7 and 1023 are not divisible by 4 (or by 16), so the scalar tail ran and still matched.